

# Análise Completa do Projeto - Sistema de Integração Live

Este é um **sistema de integração automatizada de dados de RH** que sincroniza informações entre a API eContador (Alterdata) e o sistema Hevi. Vou detalhar todos os aspectos:

---

## Estrutura Geral do Projeto

O projeto segue um padrão de **ETL (Extract, Transform, Load)**:

- **Extract:** Coleta dados da API eContador
  - **Transform:** Converte para formato CSV com regras específicas
  - **Load:** Envia via REST API ou SOAP para sistema Hevi
- 

## Módulos e Funcionalidades

### 1. config\_reader.py - Gerenciamento de Configurações



```
# Carrega configurações do arquivo .config  
  
- [APISOURCE] - Token JWT da API eContador  
  
- [APITARGET] - URL, integração, token_base para API Hevi  
  
- [SOAP] - Credenciais para webservice SOAP
```

#### Funções principais:

- ler\_config() - Lê todas as seções
  - ler\_token\_config() - Retorna token da API source
  - obter\_headers\_api() - Gera headers com Bearer token
- 

### 2. empresas.py - Integração de Empresas

#### Fluxo:

1. Carrega `lista_empresas.csv` com CNPJs e códigos legado
2. Busca cada empresa na API eContador por CNPJ
3. Gera CSV com mapeamento para sistema Hevi
4. Envia via POST para endpoint `configuracao_empresa`

#### Características:

- Usa **IDs originais da API** como código legado
- Filtra apenas empresas ativas
- Mapeia endereços e dados cadastrais

---

### 3. departamentos.py - Integração de Departamentos

#### Fluxo:

1. Extraí departamentos através dos funcionários (include relationships)
2. Mapeia departamento → empresa
3. Usa **ID do departamento** como código legado
4. Envia para `configuracao_depto`

#### Formato CSV:



---

### 4. cargos.py - Integração de Cargos/Funções

#### Fluxo:

1. Extraí funções únicas do campo `nomefuncao` dos funcionários
2. Parse do formato: "000108 - VENDEDOR"
3. Separa código e nome da função
4. Envia para `configuracao_cargo`

### Características:

- Código extraído do nome da função
  - Sem CBO (campos vazios)
- 

## 5. funcionarios.py - Integração de Funcionários ★

### Fluxo completo:

1. Busca **apenas funcionários ATIVOS** (`filter[status]=ativo`)
2. Para cada funcionário:
  - Busca empresa relacionada
  - Busca departamento relacionado
  - **Formata matrícula composta:** `{cod_empresa}b{matricula_6_digitos}`
    - Exemplo: empresa 168, matrícula 2 → 168b000002
3. Formata CPF com 11 dígitos (zeros à esquerda)
4. Mapeia 40+ campos incluindo dados pessoais e organizacionais
5. Envia para `funcionario_cadastrar`

### Campos importantes:

- `matricula`: Matrícula composta
  - `cpf`: 11 dígitos
  - `login`: CPF
  - `senha`: "Ponto123" (padrão)
  - `cracha`: CPF
  - `timezone`: America/Manaus
- 

## 6. afastamentos.py - Integração de Afastamentos

### Fluxo:

1. Busca funcionários com afastamentodescricao preenchida
2. **Exclui férias** (processadas em módulo separado)
3. Extrai datas dos campos corretos:
  - attributes[ 'afastamento' ] = Data início
  - attributes[ 'retorno' ] = Data fim
4. Define ID-AFASTAMENTO baseado na descrição:
  - Férias → 1011 (não processado aqui)
  - Atestado → 1012
  - Outros → 1012 (default)
5. Usa **matrícula composta**
6. Envia para ponto\_afastamento

#### Formato CSV:



id-afastamento;dtinicio;dtfim;obs;campo\_chave;matricula

---

## 7. ferias.py - Integração de Férias

### Fluxo específico:

1. Filtra **apenas registros com "ferias"** em afastamentodescricao
2. ID-AFASTAMENTO fixo: **1011**
3. Estima período de 30 dias se datas não disponíveis
4. Mesma estrutura de CSV que afastamentos
5. Envia para ponto\_afastamento

### Diferencial:

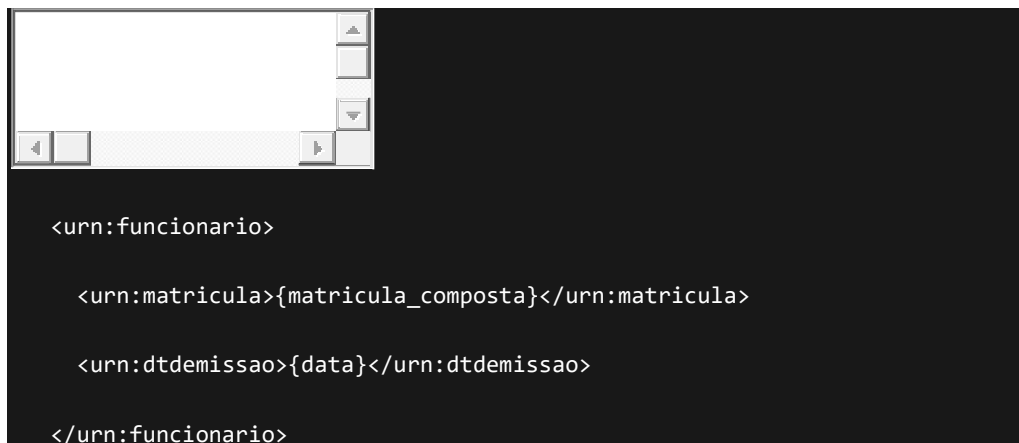
- Processamento isolado de férias
- Estimativa inteligente de 30 dias

---

## 8. demissoes.py - Integração de Demissões (SOAP) 🔧

### Fluxo complexo:

1. Busca funcionários com `filter[status]=demitido`
2. Filtra demissões desde 01/01/2025
3. Calcula datas baseadas na data real de demissão:
  - Data aviso: 30 dias antes
  - Último dia trabalhado: data demissão
  - Data acerto: 10 dias depois
4. Gera CSV para registro
5. **Envia via SOAP XML** (não REST):



6. Salva XMLs de request/response em `logs_demissao/`
7. Analisa resposta SOAP para confirmar sucesso

### Características únicas:

- Único módulo que usa SOAP em vez de REST
- Salvamento de logs XML
- Análise inteligente de resposta SOAP

---

## 9. main.py - Orquestrador Principal

## Funcionalidades:

- Banner visual com sequência de execução
- Verificação de pré-requisitos:
  - Arquivo .config
  - Módulos Python necessários
  - Token válido
- Execução sequencial dos módulos
- Pausas entre módulos (3 segundos)
- Geração de relatório JSON detalhado
- Estatísticas completas de execução

## Sequência de execução:



1. Funcionários (base para outros módulos)
2. Afastamentos
3. Demissões

**Observação:** O código comenta empresas, departamentos e cargos, mas a estrutura original previa:



1. Empresas
2. Departamentos
3. Cargos
4. Funcionários
5. Afastamentos/Férias
6. Demissões

---

## Sistema de Autenticação

### API Source (eContador)



```
headers = {  
  
    'Content-Type': 'application/vnd.api+json',  
  
    'Authorization': f'Bearer {token}'  
  
}
```

- Token JWT fixo no arquivo .config
- Válido por período específico

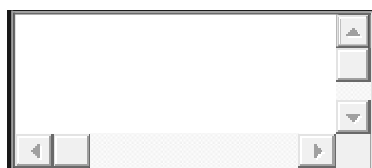
### API Target (Hevi)



```
# Token dinâmico gerado com data atual  
  
token_base = "haZZ0sJikNA8JqeRy347"  
  
data = datetime.now().strftime('%d/%m/%Y') # Exemplo: 21/11/2025  
  
concatenado = token_base + data  
  
token = hashlib.sha256(concatenado.encode('utf-8')).hexdigest()  
  
headers = {  
  
    "user": "gotech",  
  
    "token": token  
  
}
```

- Token muda diariamente
- Timezone: America/Sao\_Paulo

### SOAP (Demissões)



```
<urn:clientId>gotech</urn:clientId>

<urn:user>gotech</urn:user>

<urn:pass>G@te(ch)j83)4rZs</urn:pass>
```

---



## Padrões e Convenções

### Matrícula Composta



```
def formatar_matricula_composta(cod_empresa, matricula_original):

    matricula_6_digitos = str(matricula_original).zfill(6)

    return f"{cod_empresa}b{matricula_6_digitos}"

# Exemplos:

# empresa=168, matrícula=2    → 168b000002

# empresa=170, matrícula=1234 → 170b001234
```

### Formato CSV

- Separador: ; (ponto e vírgula)
- Encoding: **UTF-8-SIG** (com BOM)
- Headers: **lowercase**
- Datas: **DD/MM/YYYY**

### Nomenclatura de Arquivos

- empresas\_api.csv
- departamentos\_api.csv
- cargos\_api.csv
- funcionarios\_api.csv
- afastamentos\_api.csv



- ferias\_api.csv
  - demissoes\_api.csv
- 

## Pontos de Atenção e Problemas Potenciais

1. **Token Source expira:** O JWT no .config precisa ser renovado periodicamente
  2. **Dependência de ordem:** Funcionários devem ser importados antes de afastamentos/férias
  3. **IDs vs Códigos Legado:** Algumas entidades usam ID da API, outras usam códigos customizados
  4. **Estimativa de datas:** Férias e afastamentos estimam datas quando não disponíveis na API
  5. **Matrícula composta:** Requer que empresa já esteja cadastrada no sistema destino
  6. **Encoding Windows:** No main.py há `chcp 65001` para corrigir encoding no Windows
  7. **Rate limiting:** Pausas de 0.2-1 segundo entre requisições para não sobrecarregar APIs
  8. **CPF como fallback:** Usado em vários campos (login, crachá, PIS se vazio)
  9. **Campo cod\_empresa:** Funcionários mantêm campo separado da matrícula composta
  10. **Paginação:** Todas as consultas implementam paginação correta com `links.next`
- 

## Melhorias Sugeridas

1. **Validação de dados:**
  - Validar CPFs (dígitos verificadores)
  - Validar datas (não pode ser futura)
  - Validar CNPJs

## 2. **Tratamento de erros:**

- Retry automático em falhas de rede
- Log estruturado (JSON) em vez de print
- Separar erros recuperáveis de fatais

## 3. **Performance:**

- Paralelizar buscas de empresa/departamento
- Cache de resultados de empresa
- Processar em lotes maiores

## 4. **Segurança:**

- Não versionar .config com tokens
- Usar variáveis de ambiente
- Criptografar senhas

## 5. **Monitoramento:**

- Adicionar métricas (tempo, quantidade)
- Alertas em falhas críticas
- Dashboard de status

## 6. **Testes:**

- Testes unitários para cada módulo
- Testes de integração com APIs mockadas
- Validação de CSVs gerados

## 7. **Documentação:**

- README com instruções de setup
- Documentação de API endpoints
- Exemplos de uso

## 8. **Configuração:**

- Mover lista\_empresas.csv para .config
  - Configurar timezone por empresa
  - Parametrizar códigos de afastamento
- 



## Estatísticas do Código

- **Total de arquivos:** 11 (9 módulos + 2 configs)
  - **Linhas de código:** ~7000+
  - **APIs integradas:** 2 (REST) + 1 (SOAP)
  - **Entidades sincronizadas:** 7
  - **Dependências:** requests, pandas, pytz, configparser
- 



## Pontos Fortes do Projeto

1. ☒ Modular e bem organizado
2. ☒ Tratamento robusto de paginação
3. ☒ Logs descritivos e informativos
4. ☒ Geração de relatórios detalhados
5. ☒ Padrão consistente entre módulos
6. ☒ Configuração centralizada
7. ☒ Suporta execução isolada de cada módulo
8. ☒ Validação de pré-requisitos
9. ☒ Estimativas inteligentes de dados faltantes
10. ☒ Salvamento de XMLs para auditoria (demissões)